Hash Functions

A hash function maps keys to indices in a hash table. A good hash function distributes keys uniformly across the table to minimize collisions.

Natural Explanation

A hash function is a recipe that takes any key and produces a number in range $[0, m-1]$, where m is the table size.

The goal is to distribute keys so that collisions are rare, keeping operations $O(1)$ on average.

Simple Hash Functions

Modular arithmetic (for integer keys):

$$h(k) = k \bmod m$$

Fast and simple. Problems: - If $m = 10$ and keys are multiples of 10, all hash to the same position - Not uniformly distributed for certain key patterns

String hash (for strings):

$$h(s) = (c_0 \cdot a^0 + c_1 \cdot a^1 + \cdots + c_{n-1} \cdot a^{n-1}) \bmod m$$

where c_i is the ASCII value of the i-th character, and a is a constant (e.g., $a = 31$ or $a = 37$).

Combines all characters; different strings usually hash differently. Risk of overflow; use modular arithmetic carefully.

Universal Hashing

A family of hash functions is **universal** if, for any two distinct keys k_1 and k_2, the probability that they collide is at most $\frac{1}{m}$:

$$\Pr[h(k_1) = h(k_2)] \leq \frac{1}{m}$$

Benefit: even an adversary cannot craft a set of keys that all collide with a randomly chosen universal hash function.

Example (from number theory):

Choose a prime $p \geq \max(\text{key}, m)$ and random integer $a \in [1, p-1]$, $b \in [0, p-1]$:

$$h(k) = ((a \cdot k + b) \bmod p) \bmod m$$

This family is universal.

Simple Uniform Hashing

A hash function is **simple uniform** if every key is equally likely to hash to any position:

$$\Pr[h(k) = i] = \frac{1}{m} \quad \text{for all } i$$

This is the idealized assumption in average-case analysis. Real hash functions approximate this, but perfect uniformity is hard to achieve.

Practical Considerations

- **Avoid clustering**: two keys differing only slightly should have very different hashes
- **Avalanche effect**: a small change in input should drastically change output
- **Reproducibility**: same input always produces same output (for collision resolution and debugging)
- **Speed**: hashing should be much faster than comparing full objects

---

Related

- [[Hash Tables]]
- [[Collision Handling]]